

# Personal Assistant AI — Guia de Replicação

---

Guia definitivo, passo a passo, para montar do zero um assistente pessoal que funciona pelo WhatsApp — organiza agenda, tarefas e finanças automaticamente, entendendo mensagens de texto ou áudio. Escrito para quem **não tem conhecimento técnico**. Siga a ordem, sem pular partes.

Este documento cobre a versão **1.0.0** do projeto (roadmap completo).

---

## 1. Visão Geral

---

O assistente recebe mensagens pelo **WhatsApp** (texto ou áudio), interpreta a intenção usando o **Google Gemini**, transforma isso em uma instrução estruturada e executa a ação correspondente:

- Agendar, alterar ou cancelar compromissos (inclusive por referência indireta: “muda essa reunião pra 16h”)
- Criar tarefas com prazo
- Registrar gastos e entradas, e consultar saldo por período
- Gerar relatórios sob demanda (diário, semanal, mensal) e recebê-los automaticamente
- Mostrar um painel visual com indicadores (link único)
- Enviar lembretes automáticos de compromissos e tarefas
- Avisar o dono do sistema se alguma integração parar de funcionar

Tudo isso rodando sobre ferramentas gratuitas do Google (Apps Script, Sheets, Calendar, Tasks), integradas ao WhatsApp via Meta Cloud API — sem servidor próprio, sem banco de dados externo, dentro dos limites dos planos gratuitos.

---

## 2. Pré-requisitos

---

### Obrigatórios:

- Uma conta Google (Gmail) — hospeda o código (Apps Script), a planilha (Sheets), a agenda (Calendar) e as tarefas (Tasks).
- Uma conta no Facebook/Meta — para criar o app do WhatsApp.
- Um número de celular com WhatsApp — para receber as mensagens de teste (a própria Meta fornece um número de teste separado para enviar).
- Uma conta no GitHub — para guardar o código e o histórico de versões.

### Opcionais (mas recomendados):

- Uma conta no Postman — facilita testar chamadas de API diretamente, sem depender só do WhatsApp real.
- Google Chrome ou Edge — necessário caso queira usar a

extensão de sincronização automática entre Apps Script e GitHub (Parte 10).

- Uma conta no GitHub Codespaces — só é necessária se for configurar o deploy 100% automático via GitHub Actions (também Parte 10); sem isso, dá para publicar clicando manualmente em “Nova versão” a cada mudança.

---

### 3. Dicas

---

- Este guia é **autossuficiente**: seguindo as partes na ordem, dá para montar o projeto inteiro sozinho, sem ajuda de ninguém.
- Ainda assim, o processo fica bem mais rápido se você tiver acesso a um assistente de IA (como o Claude) para gerar o código de cada arquivo, revisar erros e ajudar a diagnosticar problemas ao longo do caminho. Se você tiver essa opção, veja o documento complementar **“Guia de Uso com IA”**, que explica como estruturar essa ajuda passo a passo.
- Guarde este guia à mão durante toda a configuração — várias partes remetem a decisões e problemas comuns descritos em outras seções.
- Nunca cole tokens, chaves de API ou senhas em lugares públicos (fóruns, repositórios públicos sem necessidade, capturas de tela compartilhadas). Trate qualquer token colado em um chat de IA como exposto: depois de usá-lo, revogue e gere um novo.
- Reserve um tempo maior para a Parte 2 (WhatsApp/Meta) e a Parte 12 (número real) — são as etapas com mais telas, aprovações e detalhes específicos de cada conta.

---

### 4. Decisões a Serem Tomadas

---

Antes de começar, vale decidir conscientemente estes pontos — o guia assume as opções em **negrito**, mas todas são válidas:

| Decisão                                                                      | Opções                                                                                                                  | Impacto                                                                    |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Interpretar mensagens com IA (Gemini) <b>ou</b> usar comandos fixos          | IA entende linguagem natural; comandos fixos são mais simples de implementar, mas exigem digitar frases exatas          | Este guia assume <b>uso de IA</b>                                          |
| Aceitar apenas texto <b>ou</b> também áudio                                  | Áudio é mais cômodo, mas consome mais cota gratuita do Gemini                                                           | Este guia assume <b>texto e áudio</b>                                      |
| Publicar manualmente <b>ou</b> automatizar o deploy (GitHub Actions + clasp) | Automatizar elimina cliques repetitivos, mas exige configuração inicial mais técnica (Parte 6)                          | <b>Opcional</b> — o guia funciona sem isso                                 |
| Testar com o número de teste da Meta <b>ou</b> já usar um número real        | Número de teste é gratuito e rápido, mas expira token a cada 24h e tem restrição de envio para BR/ID; número real exige | Este guia começa pelo <b>número de teste</b> e detalha a troca na Parte 12 |

|                                                                                                          |                                                                              |                                                                              |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------|
|                                                                                                          | verificação e pode ter custo por conversa                                    |                                                                              |
| Deixar o painel ( ? <code>painel=1</code> ) com link único <b>ou</b> adicionar alguma proteção de acesso | Nesta versão não há senha/login — quem tiver o link acessa dados financeiros | Este guia assume <b>link único, sem proteção adicional</b> — não compartilhe |
| Ativar a checagem diária de saúde do sistema (HealthCheck) <b>ou</b> dispensá-la                         | Sem ela, uma integração pode parar de funcionar sem ninguém perceber         | Este guia assume <b>ativada</b>                                              |

## Parte 1 — Criar a Planilha e o Projeto no Google Apps Script

1. Acesse [sheets.google.com](https://sheets.google.com) → **Planilha em branco**.
2. Renomeie a planilha para `Personal Assistant AI - DB`.
3. Renomeie a aba “Página1” para `Logs`.
4. Na linha 1, preencha as colunas: `Timestamp | Tipo | Origem | Mensagem | Status`.
5. No menu: **Extensões → Apps Script**.
6. Renomeie o projeto para `Personal Assistant AI`.

Esse projeto do Apps Script é onde todo o código vai morar, e a planilha funciona como banco de dados.

## Parte 2 — Criar o App do WhatsApp (Meta for Developers)

1. Acesse [developers.facebook.com](https://developers.facebook.com) → **Meus Apps → Criar App**.
2. Escolha o caso de uso com ícone do WhatsApp (ex: “Conectar-se com os clientes pelo WhatsApp”) → **Avançar**.
3. Se pedir um **Portfólio Comercial**: crie um novo, preenchendo nome, seu nome e e-mail comercial.
4. Revise a tela de requisitos → **Avançar** → confirme em **“Criar aplicativo”**.
5. No painel do app: **Casos de uso → Conectar-se com os clientes pelo WhatsApp → Configuração da API**. Anote (não compartilhe):
  - **Token de acesso temporário** (válido por 24h)
  - **Phone Number ID**
  - O **número de telefone de teste** fornecido pela Meta

△ O número de teste padrão da Meta é registrado nos EUA. Mensagens enviadas *dele* para números do Brasil ou Indonésia podem ser bloqueadas — isso não afeta o recebimento de mensagens que você envia para o número de teste, só o envio automático em sentido contrário (resolve-se ao trocar para um número real, Parte 12).

## Parte 3 — Conectar o Webhook do WhatsApp

Esta parte só é concluída depois que o código do projeto (Parte 7) já estiver publicado como Web App (Parte 9) — siga o guia na ordem e volte aqui quando chegar lá.

1. No painel do app: **WhatsApp → Configuração → seção Webhook → Editar.**
2. **URL de callback:** a URL do seu Web App (gerada na Parte 9).
3. **Token de verificação:** escolha uma palavra própria (ex: `meuassistente2026`) e use exatamente o mesmo valor na constante `VERIFY_TOKEN` do `Code.gs`.
4. **Verificar e salvar.**
5. No campo **“messages”**, clique em **Assinar**.

⚠ Mesmo com tudo assinado, os webhooks podem não chegar se o app não estiver “inscrito” na Conta do WhatsApp Business (WABA) correta — veja a solução na Parte 19 (Problemas Comuns).

---

## Parte 4 — Gerar a Chave do Gemini

---

1. Acesse [aistudio.google.com](https://aistudio.google.com) e faça login com a mesma conta Google do projeto.
2. Menu esquerdo → **“Get API key” → “Create API key” → “Create API key in new project”.**
3. Copie a chave gerada.

---

## Parte 5 — Configurar as Credenciais no Apps Script

---

No editor do Apps Script: engrenagem ⚙ **“Configurações do projeto” → “Propriedades do script” → “Adicionar propriedade do script”**. Cadastre uma por uma:

| Propriedade                           | Valor                                                                      |
|---------------------------------------|----------------------------------------------------------------------------|
| <code>GEMINI_API_KEY</code>           | Chave da Parte 4                                                           |
| <code>WHATSAPP_TOKEN</code>           | Token de acesso da Parte 2                                                 |
| <code>WHATSAPP_PHONE_NUMBER_ID</code> | Phone Number ID da Parte 2                                                 |
| <code>NUMERO_PROPRIETARIO</code>      | Seu número de WhatsApp (com DDI), para receber alertas críticos do sistema |

Essas propriedades nunca ficam no código — o próprio Apps Script guarda esses valores de forma protegida.

---

## Parte 6 — Sincronização Automática com o GitHub (opcional/avançada)

---

Pule esta parte se preferir colar o código manualmente (Parte 7). Ela é útil para quem vai editar o projeto com frequência.

### Extensão do navegador (2 cliques por alteração)

1. Instale, na Chrome Web Store, a extensão **“Google Apps Script GitHub Assistant”**.
2. Ative a **Google Apps Script API** em <https://script.google.com/home/usersettings>.
3. No GitHub: **Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic)**, com os escopos `repo` e `gist`. Copie o token na hora — ele só aparece uma vez.
4. No editor do Apps Script, clique em **“Login to SCM”** e informe seu usuário do GitHub + o token gerado.
5. Clique em **Repository →** selecione seu repositório → **Branch →** `main`.
6. A cada alteração de código: botão **Push ↑** → escreva a mensagem de commit → confirme.

## Deploy 100% automático (GitHub Actions + clasp)

Para eliminar até o clique no Push, é possível configurar um pipeline que publica o código automaticamente a cada push no GitHub. Resumo do processo (mais técnico):

1. Crie `.github/workflows/deploy.yml` no repositório com um job que instala o `clasp` e roda `clasp push --force` a cada push na branch `main`.
2. Gere as credenciais do `clasp` (via GitHub Codespaces, sem instalar nada localmente): `npm install -g @google/clasp`, `clasp login --no-localhost`, depois `clasp clone SEU_SCRIPT_ID`.
3. Salve o conteúdo gerado (`~/.clasprc.json` e `.clasp.json`) como dois *secrets* do repositório: `CLASPRC_JSON` e `CLASP_JSON`. Prefira configurá-los via chamada direta à API do GitHub (endpoint de secrets, criptografando com a chave pública do repositório) — colar manualmente na caixa de texto do GitHub costuma corromper o JSON por causa de quebras de linha.
4. Teste com qualquer commit e acompanhe em <https://github.com/SEU-USUARIO/SEU-REPOSITARIO/actions>.

⚠ **Cuidado:** `clasp push --force` sobrescreve o projeto inteiro, incluindo o `appsscript.json`. Se você ativar um serviço avançado (Calendar, Tasks) manualmente pela interface do Apps Script sem levar essa ativação para o `appsscript.json` do repositório, o próximo deploy automático a remove silenciosamente (sintoma: `ReferenceError: Calendar is not defined`).

💡 Mesmo com tudo automatizado, o clique em **“Nova versão”** na implantação (Parte 9) continua manual — é uma limitação do próprio Google, não tem como automatizar.

---

## Parte 7 — Colocar o Código no Projeto (caso pule a Parte 6)

---

Se você não configurou a sincronização automática, crie cada arquivo abaixo manualmente no editor do Apps Script (**Arquivo → Novo → Script**, ou **HTML** para o painel) e cole o conteúdo correspondente. Veja a função de cada arquivo na Parte 18 (Estrutura do Projeto).

Se estiver usando um assistente de IA para ajudar (ver Parte 3 — Dicas), peça para ele gerar o conteúdo de cada arquivo, um de cada vez, com base na descrição de cada etapa deste guia — é o caminho mais rápido e reduz erros de digitação.

△ Dois arquivos não podem ter o mesmo nome base no Apps Script — por exemplo, `Dashboard.gs` e `Dashboard.html` colidiriam; por isso o template HTML do painel se chama `Painel.html`.

---

## Parte 8 — Ativar os Serviços do Google (Calendar e Tasks)

---

1. No editor do Apps Script: menu “**Serviços**” → “+”.
2. Adicione “**Tasks API**”.
3. Repita para “**Calendar API**”.

Sempre que o código passar a usar um serviço novo (Calendar, Tasks, ou chamadas externas via `UrlFetchApp`), o Apps Script vai pedir uma nova autorização de permissões na primeira execução — aceite a tela que aparece (Avançado → Acessar app → Permitir).

△ Depois de ativar um serviço pela interface, copie o bloco `enabledAdvancedServices` gerado em `appscript.json` para o repositório (se estiver usando sincronização automática) — veja o aviso na Parte 6.

---

## Parte 9 — Publicar o Assistente (Web App)

---

1. **Implantar** → **Nova implantação** → **engrenagem** ⚙ → “**App da Web**”.
2. Descrição: algo identificável, ex. `Webhook - recepcao de mensagens do WhatsApp`.
3. Executar como: `Eu`. Quem pode acessar: `Qualquer pessoa`.
4. **Implantar** → autorizar acesso.
5. Copie a **URL do app da Web** — é a URL que vai na Parte 3 (Webhook) e que, com `?painel=1` no final, abre o painel visual.

A partir daqui, toda vez que qualquer arquivo `.gs` mudar, é preciso gerar uma **nova versão** da implantação para a mudança entrar no ar: **Implantar** → **Gerenciar implantações** → **lápis (editar)** → “**Nova versão**” → escreva uma descrição breve do que mudou → **Implantar**. A URL do Web App não muda.

---

## Parte 10 — Configurar os Gatilhos Automáticos (lembretes e checagem de saúde)

---

1. No editor do Apps Script: ícone de **relógio (Gatilhos)** no menu lateral → “+ **Adicionar gatilho**”.
2. Primeiro gatilho — lembretes e relatórios automáticos:
  - Função: `verificarLembretes`
  - Fonte do evento: `Baseado em tempo`

- Tipo: `Timer de minutos` → a cada `15 minutos`
- Segundo gatilho — checagem diária de saúde:
    - Função: `verificarSaudeSistema`
    - Fonte do evento: `Baseado em tempo`
    - Tipo: `Timer diário` → escolha uma janela tranquila, ex: `7h às 8h da manhã`
  - Salvar** os dois.

---

## Parte 11 — Teste Geral do Assistente

---

Envie, pelo WhatsApp, uma mensagem para cada funcionalidade e confirme o resultado:

| Mensagem de exemplo                                                 | O que confirmar                                                                |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------|
| “Marca uma reunião com o cliente amanhã às 15h”                     | Confirmação no WhatsApp + evento no Google Calendar                            |
| “Muda essa reunião pra 16h” (logo em seguida, sem repetir o título) | Assistente reconhece o compromisso pela memória de curto prazo                 |
| “Cria uma tarefa para revisar o contrato até sexta”                 | Tarefa aparece em <code>tasks.google.com</code> , na lista dedicada do projeto |
| “Gastei 50 reais com almoço”                                        | Linha nova na aba Financeiro + confirmação com categoria                       |
| “Como está meu financeiro esse mês?”                                | Resumo de entradas, gastos e saldo por WhatsApp                                |
| “Me manda o relatório da semana”                                    | Relatório combinando agenda, tarefas e financeiro                              |
| Abrir <code>SUA_URL_DO_WEB_APP?painel=1</code> no navegador         | Painel com indicadores e gráfico de gastos                                     |
| Aguardar ~1h antes de um compromisso agendado                       | Lembrete automático chega por WhatsApp                                         |
| Rodar <code>verificarSaudeSistema</code> manualmente pelo editor    | 4 linhas novas na aba Logs ( <code>HEALTHCHECK</code> ), uma por integração    |

✓ Critério de conclusão: todos os itens acima funcionando, sem necessidade de repetir informações que o assistente já deveria “lembrar”.

---

## Parte 12 — Trocar para um Número Real

---

O número de teste da Meta é ótimo para validar tudo, mas tem limitações (expira token a cada 24h, restrição de envio para BR, limite de destinatários). Para colocar em uso real:

- Registrar o número real:** no painel do app, **WhatsApp → Configuração da API → Adicionar número de telefone**. O número não pode estar em uso no aplicativo comum do WhatsApp — remova-o de lá antes, se for o caso.
- Verificar o número:** a Meta envia um código por SMS ou chamada de voz.
- Configurar o Perfil Comercial (Business Profile):** nome de exibição, categoria e descrição do negócio — o nome de exibição

passa por uma aprovação da Meta, que pode levar de horas a poucos dias.

4. **Gerar um token de acesso permanente** (elimina a expiração de 24h): no Business Manager, crie um **Usuário do Sistema** com papel Admin, atribua a ele o app do WhatsApp, e gere um token de acesso do sistema sem expiração, com os escopos `whatsapp_business_messaging` e `whatsapp_business_management`.
5. **Atualizar as Propriedades do Script**: troque `WHATSAPP_TOKEN` pelo token permanente e `WHATSAPP_PHONE_NUMBER_ID` pelo ID do número real (aparece na mesma tela de Configuração da API).
6. **Reinscrever o app no webhook** para a WABA do número real — repita a checagem/inscrição de `subscribed_apps` descrita na Parte 19, agora usando o WABA ID do número novo.
7. **Revisar a janela de conversa de 24h**: contas novas começam com um limite diário de conversas, que aumenta com o uso. Mensagens que você envia em resposta a uma mensagem do usuário (dentro de 24h) são gratuitas dentro do limite; mensagens que o **assistente inicia por conta própria** fora dessa janela (como os lembretes automáticos) podem exigir um **modelo de mensagem (template)** pré-aprovado pela Meta. Isso afeta diretamente o `Lembretes.gs` — em fase de teste, a janela costuma estar sempre aberta porque você mesmo inicia a conversa; em produção real, avalie se os lembretes proativos precisam de um template aprovado.
8. **Refazer o teste geral** (Parte 11) já com o número real.

---

## 17. Funcionalidades da Versão 1.0.0

---

- 📅 Agendar, alterar e cancelar compromissos (inclusive por referência indireta, com memória de curto prazo)
- ✔ Criar tarefas com prazo (e horário, quando informado)
- 🗂 Registrar gastos e entradas, com categorização
- 💰 Consultar saldo e gastos por categoria, por período
- 📊 Gerar relatórios sob demanda e recebê-los automaticamente (diário, semanal aos domingos, mensal no fim do mês)
- 📱 Painel visual com indicadores e gráfico de gastos (link único)
- ⌚ Lembretes automáticos de compromissos (com link do Google Meet, quando aplicável) e de tarefas
- 🏥 Checagem diária de saúde das integrações, com aviso automático em caso de falha
- 🗣 Suporte a mensagens de texto e áudio

**Conhecido, ainda não implementado:** o prompt do Gemini reconhece uma intenção de “registrar anotação” e uma intenção avulsa de “lembrete”, mas o roteador do sistema ( `Code.gs` ) não trata nenhuma das duas — mensagens desse tipo recebem uma resposta genérica de “função em construção”. Ficou como decisão em aberto para uma próxima versão (implementar de fato, ou remover essas intenções do prompt para não gerar respostas confusas).

---

## 18. Estrutura do Projeto (referência)

---

| Arquivo | Função |
|---------|--------|
|         |        |



|                                                      |                                                                           |
|------------------------------------------------------|---------------------------------------------------------------------------|
| <code>Code.gs</code>                                 | Ponto de entrada (webhook) e roteador principal dos intents               |
| <code>Gemini.gs</code>                               | Interpretação de mensagens (texto/áudio) com o Gemini                     |
| <code>WhatsApp.gs</code>                             | Envio de mensagens e download de mídia (áudio) via Meta Cloud API         |
| <code>Agenda.gs</code>                               | Criação, edição e cancelamento de compromissos no Google Calendar         |
| <code>Tasks.gs</code>                                | Criação de tarefas no Google Tasks (lista dedicada)                       |
| <code>Financeiro.gs</code>                           | Registro de gastos/entradas e cálculo de saldo                            |
| <code>Relatorios.gs</code>                           | Geração de relatórios diário/semanal/mensal                               |
| <code>Dashboard.gs</code> + <code>Painel.html</code> | Painel visual (Web App com <code>? painel=1</code> )                      |
| <code>Lembretes.gs</code>                            | Lembretes automáticos e envio automático de relatórios                    |
| <code>Config.gs</code>                               | Preferências gerais e memória de curto prazo por usuário                  |
| <code>Logs.gs</code>                                 | Registro de cada movimentação do sistema e de erros críticos              |
| <code>HealthCheck.gs</code>                          | Checação periódica de saúde das integrações                               |
| <code>appsscript.json</code>                         | Manifesto do projeto (serviços avançados habilitados, fuso horário, etc.) |

## 19. Problemas Comuns e Soluções

- **Webhook verificado, mas mensagens não chegam:** o app pode não estar “inscrito” na WABA correta. Verifique com `GET https://graph.facebook.com/v20.0/{WABA_ID}/subscribed_apps` (Bearer = seu token) e, se vier vazio, repita a mesma URL com `POST`. O WABA ID pode ser diferente do Phone Number ID — descubra com `GET .../{PHONE_NUMBER_ID}?fields=whatsapp_business_account`.
- **Erro de autenticação (131005 / 190) ao enviar mensagem:** o token de acesso temporário expira a cada 24h — gere um novo na tela de Configuração da API e atualize a propriedade `WHATSAPP_TOKEN` (resolve-se de vez ao migrar para um token permanente, Parte 12).
- **`ReferenceError: Calendar is not defined` (ou `Tasks`) depois de um deploy que antes funcionava:** um serviço avançado foi ativado pela interface, mas não estava registrado no `appsscript.json` do repositório, e um deploy automático sobrescreveu o manifesto. Registre `enabledAdvancedServices` no `appsscript.json` e sincronize.
- **Datas/horários errados nos lembretes de tarefas:** o Google Sheets converte automaticamente texto que parece data/hora ( `"2026-08-03"` , `"00:35"` ) em um objeto de Data, quebrando comparações silenciosamente. Force formato de texto puro

( `setNumberFormat("@")` ) ao gravar, e trate como texto na leitura. O mesmo cuidado vale para qualquer coluna usada como chave de busca (ex: número de telefone).

- **Modelo do Gemini retorna erro 404 “model is no longer available”:** o Google descontinua modelos periodicamente — troque o nome do modelo na chamada pelo atual recomendado em `ai.google.dev`.
- **Dois arquivos com o mesmo nome no Apps Script:** `Arquivo.gs` e `Arquivo.html` colidem por terem o mesmo nome base — use nomes diferentes (ex: `Painel.html` em vez de `Dashboard.html`).
- **Secrets do GitHub Actions corrompidos ( `Expected ',' or '}'` after property value in JSON ):** colar credenciais diretamente na caixa de texto do GitHub costuma introduzir quebras de linha indevidas. Prefira configurar via chamada direta à API do GitHub, criptografando com a chave pública do repositório.
- **Restrição de país no número de teste:** mensagens enviadas pelo número de teste da Meta (registrado nos EUA) para números do Brasil ou Indonésia podem ser bloqueadas — resolve-se ao migrar para um número real (Parte 12).

---

## 20. Glossário Rápido

---

- **Webhook:** endereço que a Meta chama automaticamente sempre que chega uma mensagem nova, entregando os dados para o seu sistema.
- **Intent:** a intenção que o Gemini identifica numa mensagem (ex: “agendar compromisso”), usada pelo código para decidir o que fazer.
- **Deploy / Implantação / “Nova versão”:** processo de publicar o código atual como a versão que está de fato no ar.
- **Trigger / Gatilho:** configuração do Apps Script para rodar uma função automaticamente, em intervalos de tempo.
- **Script Properties (Propriedades do Script):** onde o Apps Script guarda valores sensíveis (chaves, tokens) fora do código.
- **WABA (WhatsApp Business Account):** a conta comercial do WhatsApp associada ao número — pode ter um ID diferente do Phone Number ID.
- **Usuário do Sistema (System User):** tipo de usuário no Business Manager da Meta usado para gerar tokens de acesso permanentes, sem depender de uma pessoa física logada.
- **clasp:** ferramenta de linha de comando do Google para sincronizar código local (ou de um pipeline) com um projeto do Apps Script.
- **Secret (GitHub):** valor sensível guardado de forma criptografada nas configurações do repositório, usado por pipelines automáticos sem aparecer no código.
- **Cota (quota):** limite de uso gratuito de uma API (Gemini, WhatsApp) dentro de um período — estourar a cota gera erros até o limite renovar.